

B02 Quick Revision

Recall the pipeline contracts before the closed-book check

Closed-book revision map

Lesson	Recall	Engineering rule
B02-L01	Script contract	One clean entry point; explicit input/output paths; no notebook-state dependency.
B02-L02	Decomposition	Separate load, transform, validate and publish so failures are diagnosable.
B02-L03	Deterministic discovery	Discover only the intended files, sort them, record lineage and reconcile each file.
B02-L04	Merge contract	Prove master-key uniqueness before a many-to-one merge; reject unexplained row multiplication.
B02-L05	File interfaces	Declare CSV/JSON/Excel parsing assumptions, output grain and re-read checks.
B02-L06	HTTP basics	Treat response status/content/JSON shape as a contract; local fixture can prove the core skill offline.
B02-L07	Failure containment	Classify failures; raise/record them; critical validation failure blocks trusted publish.
B02-L08	Logging and evidence	Save run events and machine-readable validation, not print-only reassurance.
B02-L09	Safe reruns	A second run must not silently append duplicates or corrupt prior trusted output.
B02-L10	Reproducible batch	Defend inputs, code stages, controls, outputs, manifest, rerun behavior and C3 boundary.

Failure-first recall

- If a required column is missing, stop before transform/publish and preserve the failure evidence.
- If a numeric field contains BAD/oops, do not silently coerce the row away and then certify the output.
- If a supposed master key is duplicated, stop before merge; validate=many_to_one should be a contract, not decoration.
- If row count or control total changes after a merge, explain the cardinality before trusting any aggregate.
- If a live HTTP call is unavailable, prove the parsing/shape/validation logic on the supplied offline JSON rather than inventing a successful request.
- If a rerun changes a trusted output without a declared policy, the pipeline is not yet reproducible.

Move on when

With the learner book closed, you can describe the full path - discover -> load -> validate -> transform/merge -> validate -> publish -> log/manifest - and name the exact checks that prevent a bad run from becoming a trusted output.